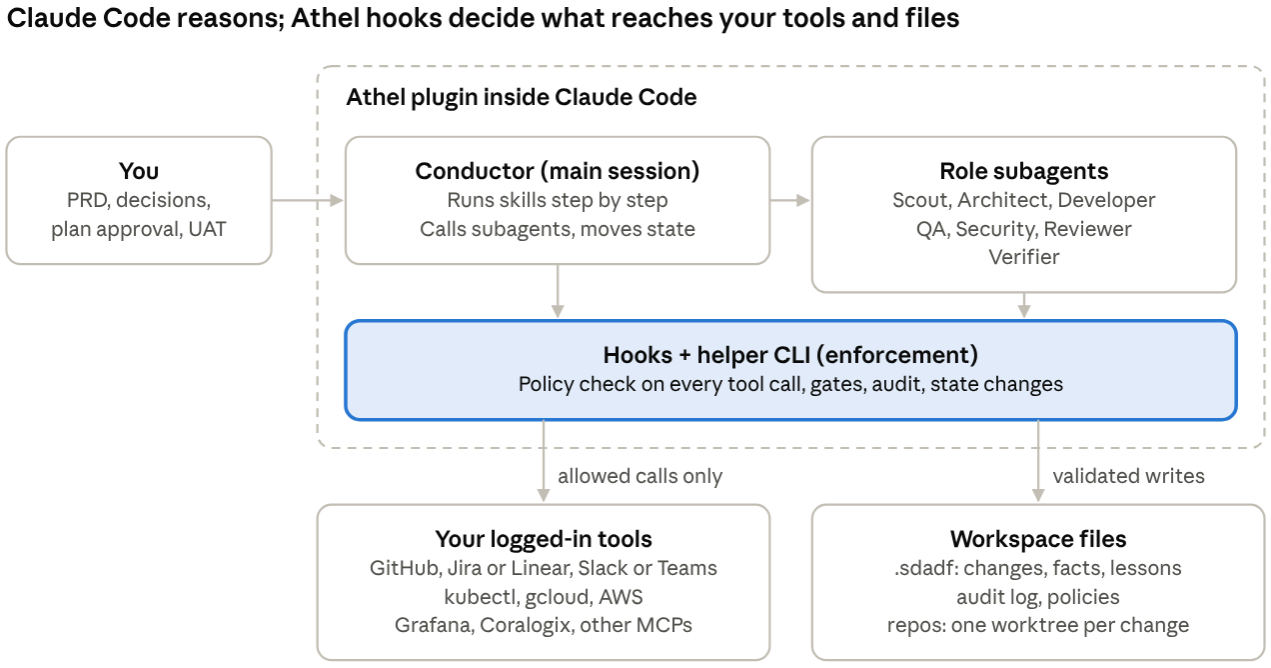
Athel — 02 Master TRD

Nov 1, 2026 · @Vaibhav Shukla

Athel is a Claude Code plugin made of skills, subagents, hooks and a small helper CLI, with all state kept as files in the workspace; this document defines that system and the rules every module follows.

Architecture at a glance



SDADF architecture · conductor, subagents, enforcement layer, tools, workspace

Every tool call and every state write from the conductor or a subagent passes through the hook layer, so no model decision can reach a real system without a policy check.

Key design decisions

Ten decisions shape everything else; each is recorded as an ADR in `docs/adr/` of the Athel repo when implementation starts.

ADR	Decision	Alternatives rejected	Reason
001	Ship as a Claude Code plugin (skills, subagents, hooks, MCP)	Standalone orchestrator calling the API; fork of an existing framework	Reuses Claude Code's agent loop, tools, permissions and auth; least code to own
002	All state as plain files in `.athel/` (YAML, Markdown, JSONL)	SQLite or a server database	Human-readable, diffable, git-friendly, survives any session; no daemon to run
003	A small deterministic helper CLI `athel` (Go, shipped as one static binary per platform inside the plugin) for state changes, locks, schema validation and policy checks	Let the model edit state files directly	Model output is probabilistic; state transitions and policy checks must be exact and testable
004	Hooks enforce safety; skills only describe behaviour	Rules written into prompts	A prompt can be ignored or injected; a hook that exits with a deny cannot
005	One subagent per role with its own tool allow-list	One agent doing everything	Isolated context per role, least privilege, independent review
006	Knowledge is discovered live and cached as verifiable facts	A maintained service catalog as the source of truth	Catalogs go stale; tools reflect reality (Master PRD principle 1)
007	Tools reached through existing MCP servers and authenticated CLIs, wrapped by Athel policy	Athel re-implements integrations	Developers already have these logged in; Athel adds policy, not plumbing
008	Every write action must map to a task in an approved plan	Free-form actions	Blocks prompt-injected or out-of-scope actions; makes every action explainable
009	Humans merge and deploy, in every environment	Auto-merge on green; Athel-triggered staging deploys	Accountability and blast-radius control in v1

ADR	Decision	Alternatives rejected	Reason
010	Git worktrees per change per repo	Switching branches in the developer's checkout	Never disturbs the developer's own work; enables parallel tasks later

Plugin layout and component responsibilities

The plugin has six kinds of component; skills hold the method, subagents do the work, hooks and the helper CLI enforce the rules.

```
athel/
├── .claude-plugin/plugin.json      name, version, min Claude Code version
├── skills/                         one folder per workflow step, each with
SKILL.md
|   ├── start/                      create a change from a PRD or request
|   ├── investigate/                live discovery playbook (doc 03)
|   ├── spec/                       PRD to specs (doc 05)
|   ├── blast-radius/               impact analysis (doc 05)
|   ├── decide/                     decision cards (doc 05)
|   ├── plan/                       change plan and task graph (doc 05)
|   ├── git-preflight/              safe branching (doc 04)
|   ├── build/                      task loop (doc 06)
|   ├── review/                     iterative review (doc 06)
|   ├── verify/                     staging verification (doc 07)
|   ├── ship/                       open linked PRs (doc 04)
|   ├── uat/                        UAT round intake (doc 07)
|   ├── learn/ · lessons/           extract, confirm and list lessons (doc
07)
|   ├── status/ · resume/           progress and recovery
|   └── integrations/<tool>/        per-tool usage playbooks (doc 03)
├── agents/                         subagents, one Markdown file per role
|   ├── scout.md · architect.md · developer.md
|   ├── qa.md · security.md · reviewer.md · verifier.md
├── hooks/hooks.json                event wiring; handlers call the helper
CLI
├── bin/athel                       helper CLI (state, locks, policy,
schemas)
├── schemas/                        JSON Schemas for every state file
├── policies/defaults/              default tools.yaml, git.yaml, gates.yaml
├── templates/                      spec, plan, decision card, PR body,
report
└── tests/                          unit, hook red-team, scenario, eval
suites
```

Component	Responsibility	Must never
Skills	Describe how to do a step, which subagent to call, which files to read and write	Contain safety rules that only they enforce
Subagents	Do one role's work in an isolated context with a minimal tool list	Change workflow state directly; call tools outside their allow-list
Hooks	Allow or deny each tool call; block stopping when gates fail; write the audit log	Depend on model judgement for a safety decision
Helper CLI	Validate and change state, take locks, evaluate policy, compute IDs and hashes	Call an LLM; make network calls except git and configured health checks
Schemas	Define every file's shape; versioned	Change without a migration
Templates	Keep outputs consistent and reviewable	Hold logic

The main Claude Code session acts as the **conductor**: it runs the current skill, calls subagents, and asks the helper CLI to move the change to its next state. Subagents return structured results (YAML blocks validated against a schema) rather than free text.

Runtime model

One change runs as a sequence of steps; each step is one skill run by the conductor, which delegates heavy work to subagents so the main context stays small and every step can resume from files.

Execution rules

1. A step starts by calling `athel state enter <step>`, which checks the transition is legal, takes the change lock and records the start.
2. The step reads only the files it needs, listed in its SKILL.md under "Inputs".
3. Heavy reading (codebases, logs, dashboards) happens in subagents. A subagent returns a summary of at most 2,000 words plus a structured result; raw data goes to `.athel/changes/<id>/artifacts/`, never into the conductor's context.
4. A step ends by writing its outputs and calling `athel state exit <step> --result ok|blocked|failed`. The helper validates outputs against schemas before accepting `ok`.

5. If the context is compacted or the session dies, `/athel:resume` reads `change.yaml` and continues from the last completed step. No step depends on chat history.

Hooks used (event names per the Claude Code hooks reference)

Event	Athel use
`SessionStart`	Load active change summary, lessons index and policy versions
`UserPromptSubmit`	Attach active change ID and current step to the prompt context
`PreToolUse`	Policy decision on every tool call: allow, deny, or require approval (doc 04)
`PostToolUse` / `PostToolUseFailure`	Audit record; scrub secrets from output; update fact access log
`SubagentStop`	Validate the subagent's structured result; block if it is missing or invalid
`Stop`	Block the conductor from finishing a step whose gates have not passed
`PreCompact`	Flush a step checkpoint to disk before compaction
`SessionEnd`	Release locks held by this session, write a session summary

`Stop` and `SubagentStop` hooks must check `stop_hook_active` (or an equivalent attempt counter in the change state) so a failing gate can never loop forever; after 3 blocked stops the step moves to `blocked` and raises a decision card.

Model and budget per role (defaults, configurable)

Role	Default model tier	Context strategy	Max turns per call
Conductor	Strongest available	Small: state, plan, summaries only	n/a
Scout	Fast tier for wide search; strongest for synthesis	Fresh per question	40
Architect	Strongest	Specs and scout summaries	30
Developer	Strongest	One task's spec, relevant files	60

Role	Default model tier	Context strategy	Max turns per call
QA	Strong	Task spec, diff, test files	40
Security	Strongest	Diff, threat checklist, dependency data	30
Reviewer	Strongest, separate from developer call	Diff, spec, standards; never the developer's reasoning	30
Verifier	Fast tier	Baseline and current metrics	20

Every change has a budget in `change.yaml` (tokens or cost, wall time, tool calls). At 80% the conductor writes a warning to the status; at 100% the change moves to `blocked` with a decision card asking to extend or stop.

State model

Everything Athel knows lives under `.athel/` in the workspace, and every file has a JSON Schema with a version number; the helper CLI rejects any write that fails its schema.

```
workspace/
├── repos/<service>/                git worktrees, one per change per repo
├── cache/<repo>.git                Athel's own partial clones; base for
worktrees (doc 03)
└── .athel/
    ├── config/                     project-level policy overrides
    ├── knowledge/facts.jsonl       verified fact cache (doc 03)
    ├── knowledge/access.jsonl      which fact was used, verified or not
    ├── lessons/                    learned rules, one file each (doc 07)
    ├── repos/<repo>/               conventions.md and map.md per repo (doc
05)
    ├── repos/index.yaml            developer's local clones mapped to
remotes (doc 03)
    ├── audit/YYYY-MM-DD.jsonl      append-only audit log (doc 04)
    └── changes/CHG-042/
        ├── change.yaml             the change's state, owner, budget,
repos
        ├── prd/v1.md, v2.md …      PRD versions (each UAT round may add
one)
        ├── investigation.md        scout findings with fact IDs
        ├── spec/<service>.md       requirements, design, acceptance
criteria
        ├── blast-radius.md         impact report (doc 05)
        ├── decisions.yaml          every decision card and its answer
```

```
        ├── traceability.md           PRD → REQ → AC → task → test links (doc
05)
        ├── preflight/<repo>.md       git preflight report per repo (doc 04)
        ├── plan.yaml                 task graph with status per task
        ├── reviews/<task>/round-N.yaml  review findings and their closure
        ├── gates/<task>.yaml         gate results with evidence links
        ├── findings.yaml             side findings (doc 07)
        ├── verify/round-N.md         staging verification (doc 07)
        ├── uat/round-N.md            human feedback
        └── artifacts/                raw outputs too large for context
```

`change.yaml` (abridged)

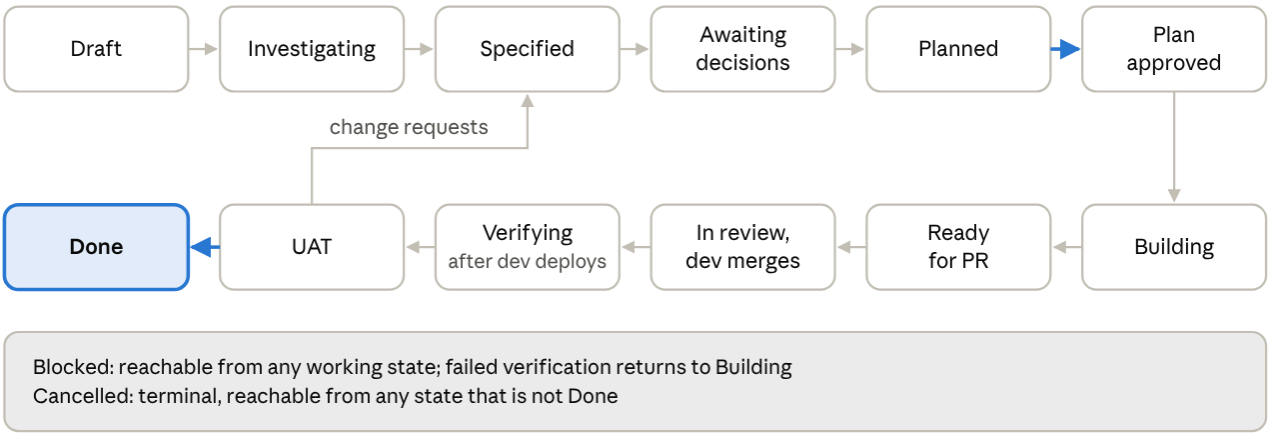
```
schema: athel/change@1
id: CHG-042
title: Customers can update phone number
state: building         # see lifecycle below
step_attempts: {build: 1}
created_by: me@example.com
repos:
  - name: user-service
    base: main
    branch: athel/CHG-042-phone-update
    worktree: repos/user-service@CHG-042
budget: {max_cost_usd: 40, max_hours: 8, used_cost_usd: 12.6}
lock: {session: abc123, since: 2026-11-01T10:12:00Z, ttl_min: 30}
history:                # append-only transitions
  - {from: planned, to: plan_approved, by: human, at: …}
```

Lifecycle states

`draft → investigating → specified → awaiting_decisions → planned → plan_approved → building → ready_for_pr → in_review → verifying → uat → done`, plus `blocked` (reachable from every working state, returns to the state it came from) and `cancelled` (terminal, from any non-terminal state). A UAT round with change requests moves the change back to `specified` with a new PRD version. The diagram below shows the allowed transitions.

A change moves forward only through checked transitions

UAT change requests send it back to spec; verification waits for the developer to deploy



Blue arrows need a human: plan approval and final acceptance. Merging and deploying are always human.

change lifecycle · 12 states, 2 human gates, 1 loop back

Only the helper CLI performs these transitions, and the two blue ones accept only a human-originated command. Verifying starts after the developer deploys (a branch deploy or the merge itself); Athel waits for it and never deploys. A failed verification sends the change back to Building.

Workflow engine

The helper CLI is the only component allowed to change a change's state, so every transition is checked, logged and repeatable.

Transition rules

- Transitions are a fixed table in the CLI. An illegal transition (for example `planned → building` without approval) exits non-zero and writes an audit record.
- Transitions that need a human (`plan_approved`, waivers, `done`) require `--by human` and are only accepted from a `UserPromptSubmit`-originated command, never from a subagent. The hook layer enforces this.
- Each transition records `from`, `to`, `by`, `at`, `reason` and the hash of the files that justified it (for example the plan hash at approval). If the plan changes after approval, the approval is void and the change returns to `planned`.

Locking

- One active writer per change. `athel lock acquire` writes the session ID and a 30-minute TTL; every tool call refreshes it.

- A second session on the same change gets read-only access and a message naming the lock holder.
- A stale lock (TTL expired, no heartbeat) can be broken with `/athel:resume --take-over`, which is logged.
- Repo-level locks prevent two changes from writing the same worktree.

Resume and idempotency

- Each step is split into checkpoints written to `change.yaml` (for example `build.task-07.review-round-2`). Resume restarts at the last checkpoint, not the step start.
- Every side-effecting operation has an idempotency key `<change>:<step>:<task>:<op>`. Before acting, the CLI checks the audit log: an operation already done is skipped, an operation started but not confirmed is re-checked against the real system (for example "does this branch already exist on the remote?").
- Git operations are designed to be safe to repeat: create branch if absent, push only fast-forward, open PR only if no PR exists for that branch.

Retries

Failure kind	Example	Policy
Transient tool error	Timeout, 5xx, rate limit	Up to 3 retries with backoff 2s, 8s, 30s
Auth error	Expired gcloud or kubectl credentials	No retry; block with "re-authenticate X"
Gate failure	Test fails	Back to developer with findings; max 5 fix rounds per task, then decision card
Invalid subagent result	Missing required field	Re-ask once with the schema error; then fail the step
Policy deny	Forbidden command	No retry; record; if the plan needs it, decision card
Unknown	Anything else	Block with full context saved to `artifacts/`

Configuration and policy files

Policy comes from four layers, and a lower layer may only make rules stricter, never looser, except where the higher layer explicitly marks a rule `overridable: true`.

Layer	Location	Who edits	Example
1. Built-in defaults	`athel/policies/defaults/`	Athel maintainers	Never force-push; never read secret values
2. Org	`~/.athel/org/` synced from an org repo (later)	Platform team	Allowed git hosts, required gates
3. User	`~/.athel/user.yaml`	The developer	Preferred models, notification style
4. Project	`.athel/config/`	Repo owners	Test command, coverage threshold, base branch

Files

File	Holds	Detailed in
`athel.yaml`	Workspace settings, budgets, model per role, ask thresholds	This doc
`tools.yaml`	Every tool and MCP server: purpose, environments, allowed verbs, approval needs	03, 04
`environments.yaml`	Names of environments and how to recognise them (kube contexts, gcloud projects, AWS accounts, URLs)	04
`git.yaml`	Branch naming, base branches, PR rules, protected branches, commit format	04
`gates.yaml`	Gate list per repo type, thresholds, which are waivable and by whom	06
`services.yaml` (optional)	Human hints: service to repo mapping, owners. Treated as facts to verify, not truth	03

Rules

- `athel config validate` runs at `SessionStart`; an invalid policy blocks all write tools until fixed.
- The effective policy is computed once per session and its hash is written into every audit record, so any action can be traced to the exact rules in force.

- Unknown tools default to **forbidden for writes, allowed for reads only after approval**. A tool must be listed in `tools.yaml` to be used freely.

Cross-cutting concerns

These rules apply to every module; a module document may add rules but never relax these.

Error taxonomy (every error carries one code, a message, the step and the next action)

Code prefix	Meaning	Default next action
`E-POLICY-*`	Blocked by policy	Record; decision card only if the plan requires the action
`E-AUTH-*`	Tool not authenticated or expired	Block; tell the human which login to refresh
`E-TOOL-*`	Tool failed or unreachable	Retry per policy, then block
`E-STATE-*`	Illegal transition, schema failure, lock conflict	Stop the step; never auto-fix state
`E-GATE-*`	A gate failed	Return to the build loop
`E-BUDGET-*`	Budget exceeded	Block with extend-or-stop card
`E-CONFLICT-*`	Someone else's work in the way	Block; never proceed over it
`E-INTEGRITY-*`	Tampered or inconsistent files (hash mismatch)	Stop everything for this change; human review

Observability of Athel itself

- Every step writes start and end events with duration, cost and tool counts to `audit/`.
- `/athel:status` renders a one-screen summary: state, current task, gates, open decision cards, budget used, blockers.
- A weekly `athel report` (later) aggregates the metrics in the Master PRD.

Compatibility

- `plugin.json` declares the minimum Claude Code version tested. `SessionStart` checks the running version and warns on untested versions.
- All Claude Code-specific details (hook payload fields, subagent frontmatter, tool names) are read in one adapter module of the helper CLI, so a Claude Code change is fixed in one place.

- A compatibility test suite runs the hook red-team tests against each new Claude Code release before upgrading.

Security baseline

- Least privilege per subagent: each agent file lists its allowed tools; the scout and reviewer have no write tools at all.
- No secret values in context, files or logs. Output from tools passes a scrubber (known key patterns, high-entropy strings, configured secret names) before it reaches the model or the audit log.
- All tool output is untrusted data. Skills say so explicitly, and hooks enforce that write actions match an approved plan task (ADR 008).
- State files are hashed at each transition; a mismatch on the next read raises `E-INTEGRITY`.

Determinism and reproducibility

- IDs, timestamps, hashes and ordering come from the helper CLI, never from the model.
- Every decision that changes code cites the spec line, decision card or lesson that caused it.

How Athel itself is tested

Athel is tested at five levels, and the safety levels must be at 100% pass before any release.

Level	What is tested	How	Pass bar
1. Unit	Helper CLI: transitions, locks, policy evaluation, schemas, scrubber, idempotency	Ordinary unit tests, no LLM	100% pass, 90% line coverage on the CLI
2. Hook red-team	Every forbidden action in docs 04 and 03, in many phrasings, including commands hidden in tool output, chained shell commands, aliases, encoded strings	Feed synthetic tool-call payloads to hooks directly	100% blocked, 0 false allows
3. Contract	Subagent result schemas, skill input/output files	Schema tests on recorded outputs	100% valid

Level	What is tested	How	Pass bar
4. Scenario	Full flows against fake services: a local git server, a fake kube API, fixture dashboards, fake Jira	Scripted runs of Claude Code in headless mode	Each scenario's expected end state reached
5. Evals	Real-ish PRDs on sample multi-service systems; graded on blast radius recall, spec quality, gate pass rate, cards asked, cost	Eval harness with rubric grading and human spot checks	No regression over the previous release

Rules

- No release if any level 1 to 3 test fails, or if level 2 coverage of the forbidden-action list drops.
- Every bug found in real use becomes a scenario or red-team case before it is fixed.
- A sample system for scenarios and evals ships in `tests/fixtures/shop/`: 5 small services (users, orders, payments, notifications, gateway) with REST calls, one event topic, a staging kube namespace definition and dashboards as JSON.

Sources and open technical questions

Open technical questions

- Decided, helper CLI language: Go. It builds to one static binary with nothing to install, starts in a few milliseconds (the PreToolUse hook runs on every tool call, so startup time matters), and has mvdan.cc/sh, a complete Bash parser, which FR-SAFE-009 needs. Go code is plain and easy to read, and most well-known CLIs (kubectl, gh, Terraform) are written in it. Python was considered: readable, but it needs a matching runtime on every machine, and startup with YAML and schema libraries can approach or exceed the 50 ms hook budget.
- Hook latency budget: every tool call passes through `PreToolUse`. Target under 50 ms per decision; needs a benchmark in milestone M0.
- Plugin-provided subagents cannot carry their own hooks, MCP servers or permission mode, so all enforcement must sit in plugin-level hooks. To confirm against the current docs at implementation time.
- Decided, headless runs are supported: Athel steps can run without an open Claude Code window through Claude Code's non-interactive mode (`claude -p`), for example a scheduled overnight investigation or a GitHub Actions job that posts a blast radius

report on every PR. Rules in headless mode: the same hooks and policy apply; every `approve` decision becomes `deny`, since nobody is there to approve; decision cards are queued in `decisions.yaml` and summarised on Athel's own PR or in the run report, never answered by defaults except `default_safe` cards; budgets are halved by default; in CI, only read-only credentials are given to the job.

Sources

- Claude Code features overview
- Claude Code plugins reference
- Claude Code hooks reference